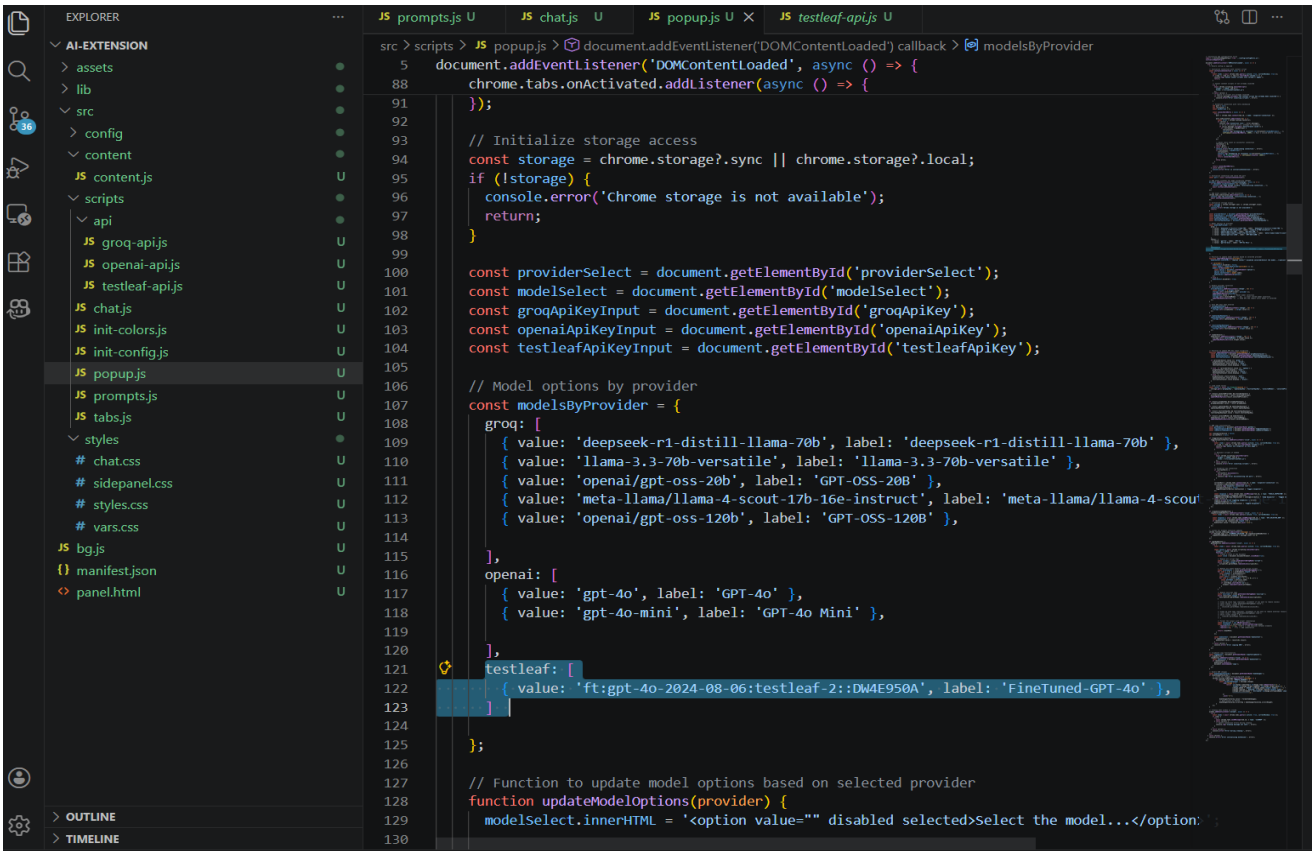
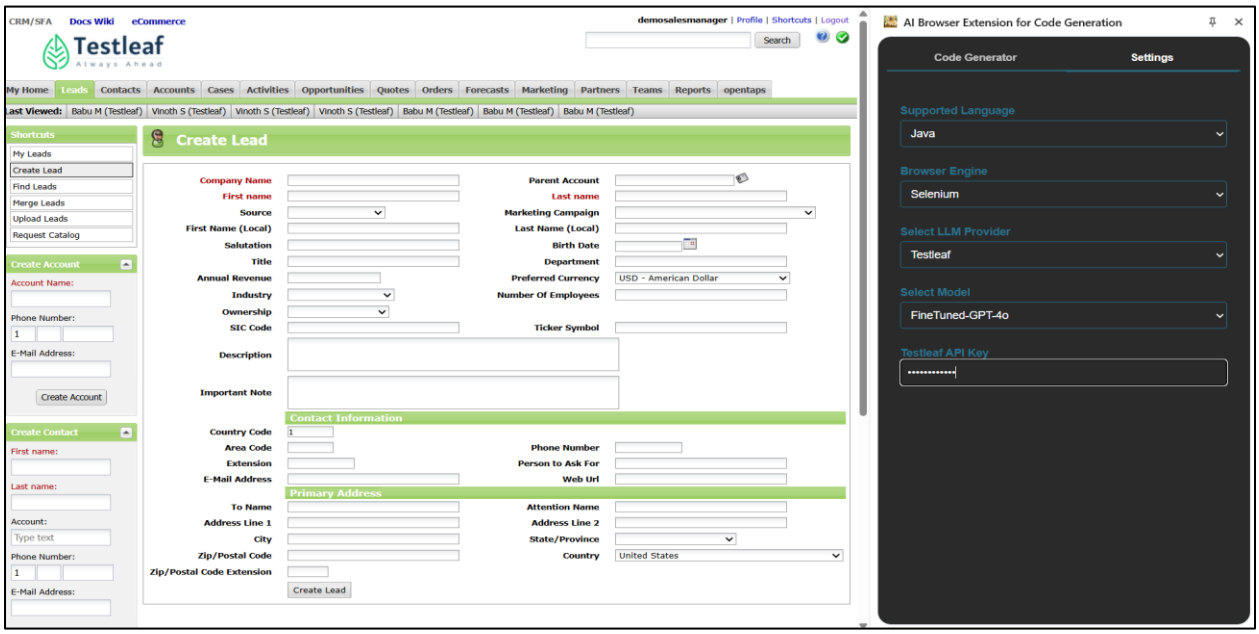
FINE TUNING

Created JSONL file and generated the fine tuned model - **ft:gpt-4o-2024-08-06:testleaf-2::DW4E950A** using the created jsonl file in week04-day01-assignment-answers.

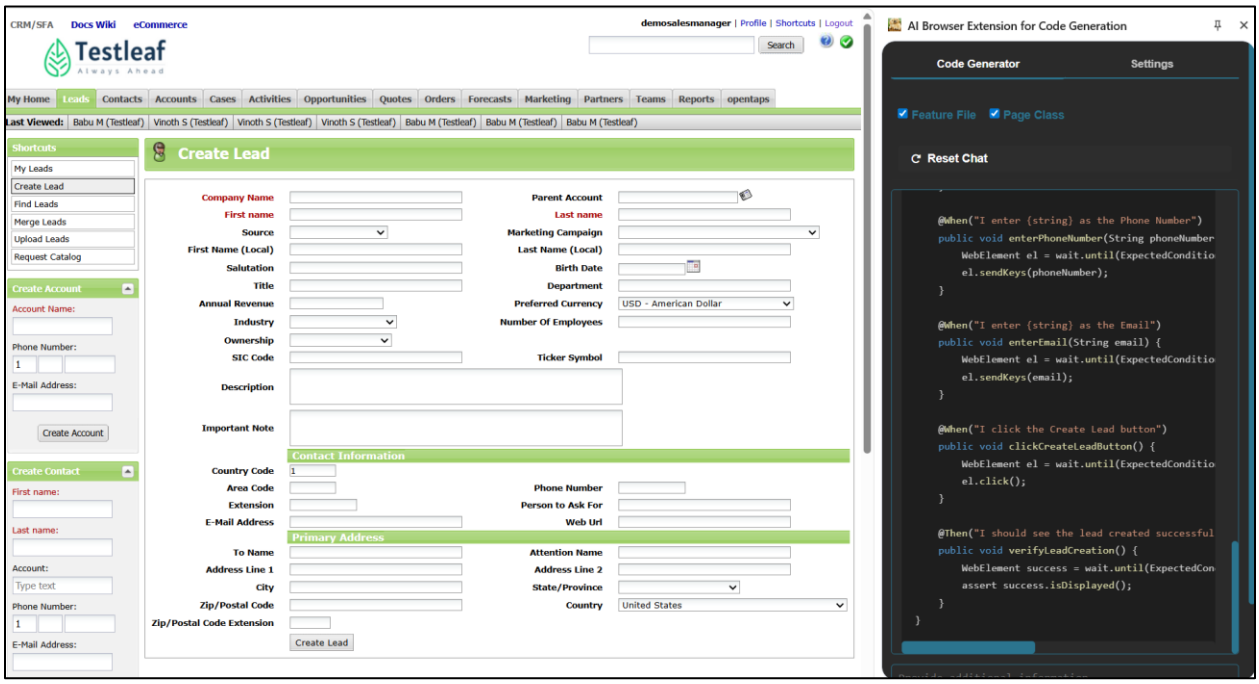
Step1: Including the Fine tuned model in popup.js of browser extension code.


EXPLORER
AI-EXTENSION
assets
lib
src
config
content
content.js
scripts
api
groq-api.js
openai-api.js
testleaf-api.js
chat.js
init-colors.js
init-config.js
popup.js
prompts.js
tabs.js
styles
chat.css
sidepanel.css
styles.css
vars.css
bg.js
manifest.json
panel.html
OUTLINE
TIMELINE
prompts.js U
chat.js U
popup.js U
testleaf-api.js U
src > scripts > popup.js > document.addEventListener('DOMContentLoaded') callback > modelsByProvider
5 document.addEventListener('DOMContentLoaded', async () => {
88 chrome.tabs.onActivated.addListener(async () => {
91 });
93 // Initialize storage access
94 const storage = chrome.storage?.sync || chrome.storage?.local;
95 if (!storage) {
96 console.error('Chrome storage is not available');
97 return;
98 }
100 const providerSelect = document.getElementById('providerSelect');
101 const modelSelect = document.getElementById('modelSelect');
102 const groqApiKeyInput = document.getElementById('groqApiKey');
103 const openaiApiKeyInput = document.getElementById('openaiApiKey');
104 const testleafApiKeyInput = document.getElementById('testleafApiKey');
106 // Model options by provider
107 const modelsByProvider = {
108 groq: [
109 { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
110 { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },
111 { value: 'openai/gpt-oss-20b', label: 'GPT-OSS-20B' },
112 { value: 'meta-llama/llama-4-scout-17b-16e-instruct', label: 'meta-llama/llama-4-scout
113 { value: 'openai/gpt-oss-120b', label: 'GPT-OSS-120B' },
115],
116 openai: [
117 { value: 'gpt-4o', label: 'GPT-4o' },
118 { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
120],
121 testleaf: [
122 { value: 'ft:gpt-4o-2024-08-06:testleaf-2::DW4E950A', label: 'FineTuned-GPT-4o' },
123]
125 };
127 // Function to update model options based on selected provider
128 function updateModelOptions(provider) {
129 modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option</option></option></option>

Step2: Selecting the Fine tuned model in AI Browser Extension for Selenium/Java code



Step3: Generating the Page class code in AI browser extension using the Fine tuned model for Leaftaps Create Lead functionality



Step4: Selenium/Java page class code with the fine tuned model - ft:gpt-4o-2024-08-06:testleaf-2::DW4E950A for leaftaps CreateLead functionality

Feature: Create Lead in OpenTaps

Scenario Outline: Successful creation of a lead with valid details

Given I open the create lead page

When I enter "<companyName>" as the Company Name

And I enter "<firstName>" as the First Name

And I enter "<lastName>" as the Last Name

And I select "<source>" as the Source

And I select "<marketingCampaign>" as the Marketing Campaign

And I enter "<phoneNumber>" as the Phone Number

And I enter "<email>" as the Email

And I click the Create Lead button

Then I should see the lead created successfully

Examples:

| companyName | firstName | lastName | source | marketingCampaign | phoneNumber | email |

| "TCS" | "Arun" | "Kumar" | "Cold Call" | "Affiliate Sites" | "9876543210"| "arun.kumar@tcs.com"|

| "Infosys" | "Lakshmi" | "Menon" | "Conference"| "Automobile" | "8765432109"| "lakshmi.m@infosys.com"|

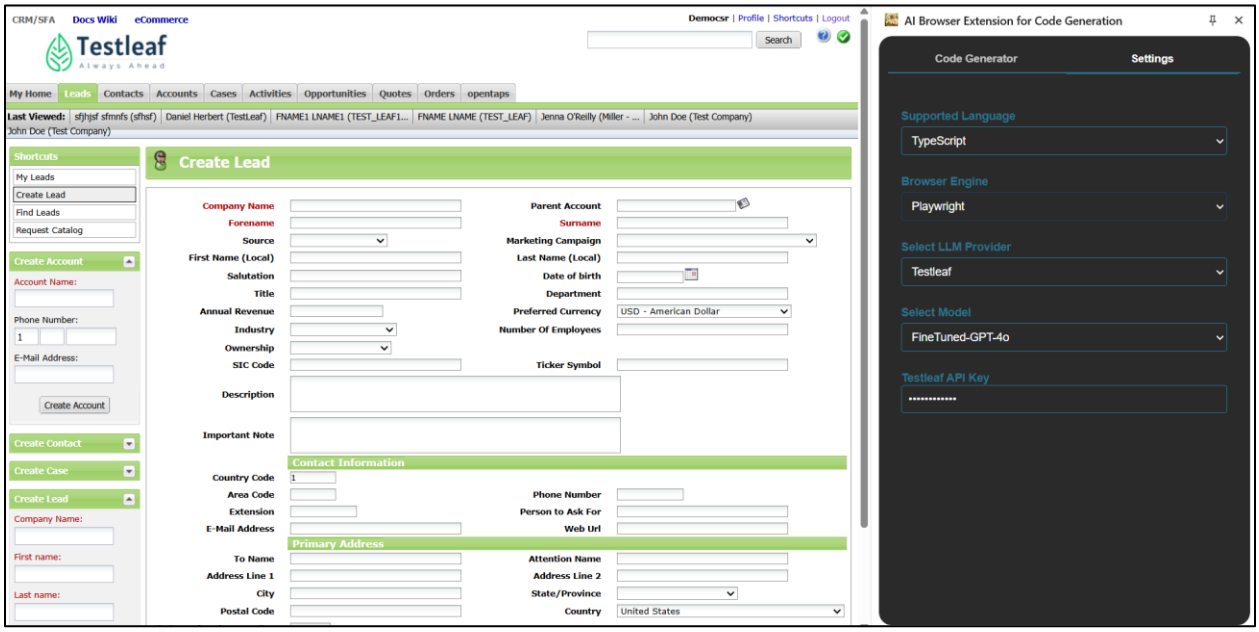
```
package com.leaftaps.stepdefs;

import io.cucumber.java.en.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;
```

```
import java.time.Duration;

public class CreateLeadStepDefinitions {
   private WebDriver driver;
   private WebDriverWait wait;

   @io.cucumber.java.Before
   public void setUp() {
      driver = new ChromeDriver();
      wait = new WebDriverWait(driver, Duration.ofSeconds(10));
      driver.manage().window().maximize();
   }

   @io.cucumber.java.After
   public void tearDown() {
      if (driver != null) driver.quit();
   }

   @Given("I open the create lead page")
   public void openCreateLeadPage() {
      driver.get("https://leaftaps.com/crmsfa/control/createLeadForm");
   }

   @When("I enter {string} as the Company Name")
   public void enterCompanyName(String companyName) {
      WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_companyNa
me")));
      el.sendKeys(companyName);
   }

   @When("I enter {string} as the First Name")
```

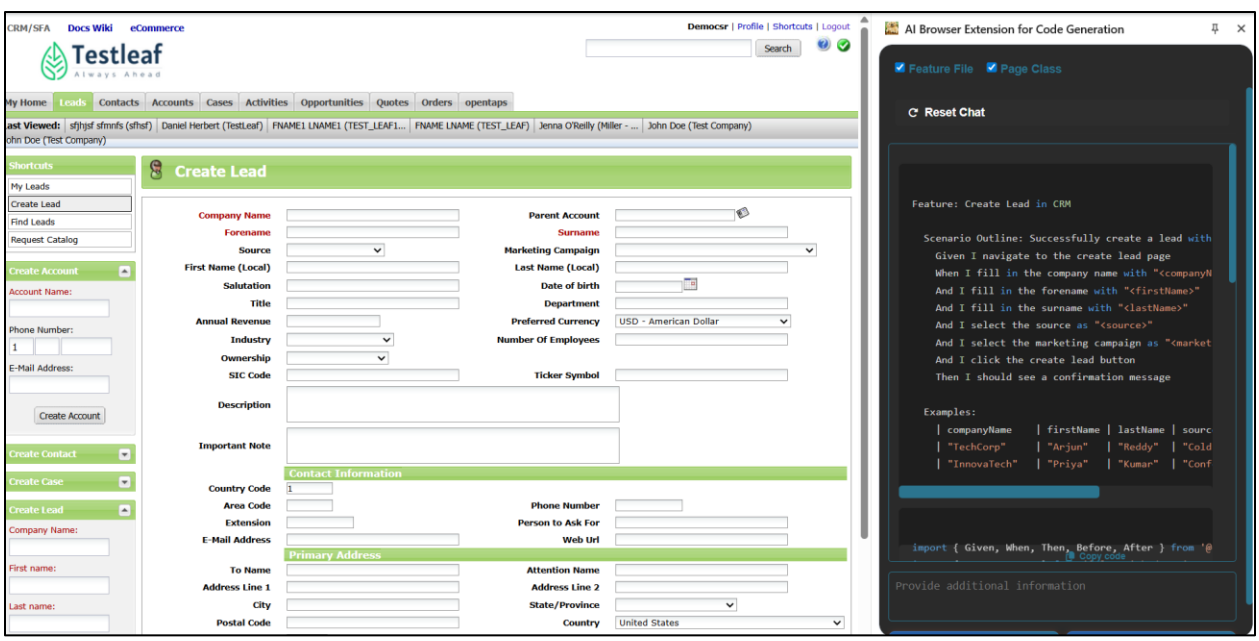
```
  public void enterFirstName(String firstName) {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_firstName"))
);
    el.sendKeys(firstName);
  }

  @When("I enter {string} as the Last Name")
  public void enterLastName(String lastName) {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_lastName"))
);
    el.sendKeys(lastName);
  }

  @When("I select {string} as the Source")
  public void selectSource(String source) {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_dataSourceI
d")));
    new Select(el).selectByVisibleText(source);
  }

  @When("I select {string} as the Marketing Campaign")
  public void selectMarketingCampaign(String marketingCampaign) {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_marketingC
ampaignId")));
    new Select(el).selectByVisibleText(marketingCampaign);
  }

  @When("I enter {string} as the Phone Number")
  public void enterPhoneNumber(String phoneNumber) {
```

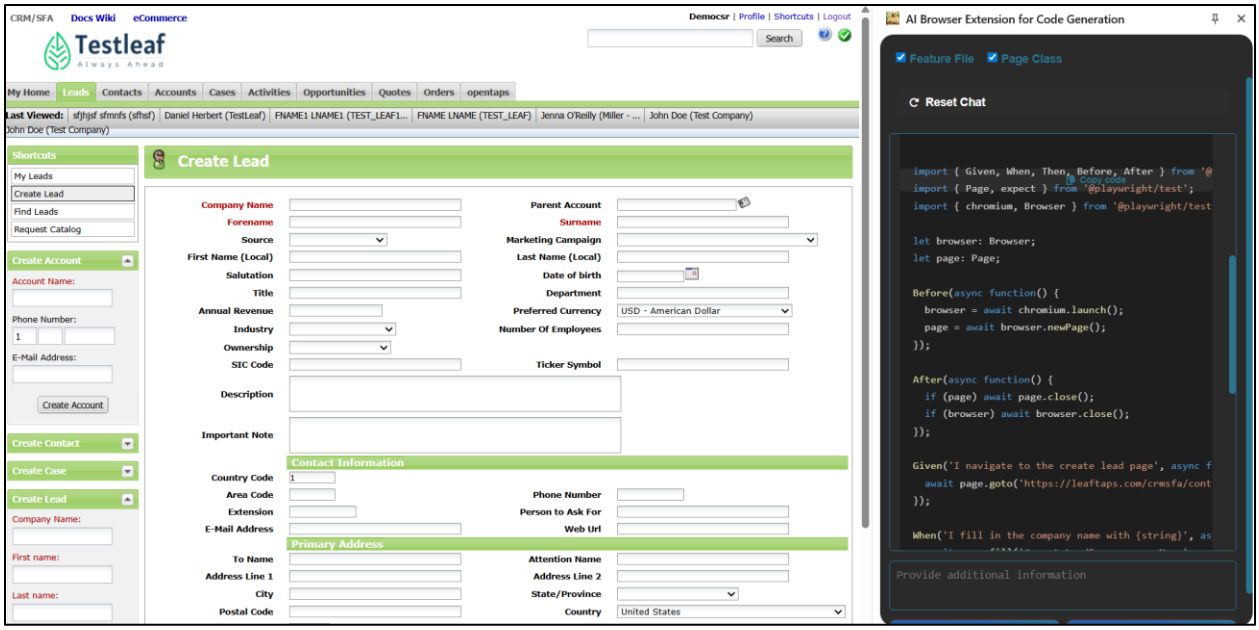
```
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_primaryPho
neNumber")));
    el.sendKeys(phoneNumber);
  }

  @When("I enter {string} as the Email")
  public void enterEmail(String email) {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.id("createLeadForm_primaryEma
il")));
    el.sendKeys(email);
  }

  @When("I click the Create Lead button")
  public void clickCreateLeadButton() {
    WebElement el =
wait.until(ExpectedConditions.elementToBeClickable(By.name("submitButton")));
    el.click();
  }

  @Then("I should see the lead created successfully")
  public void verifyLeadCreation() {
    WebElement success =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("success")));
    assert success.isDisplayed();
  }
}
```

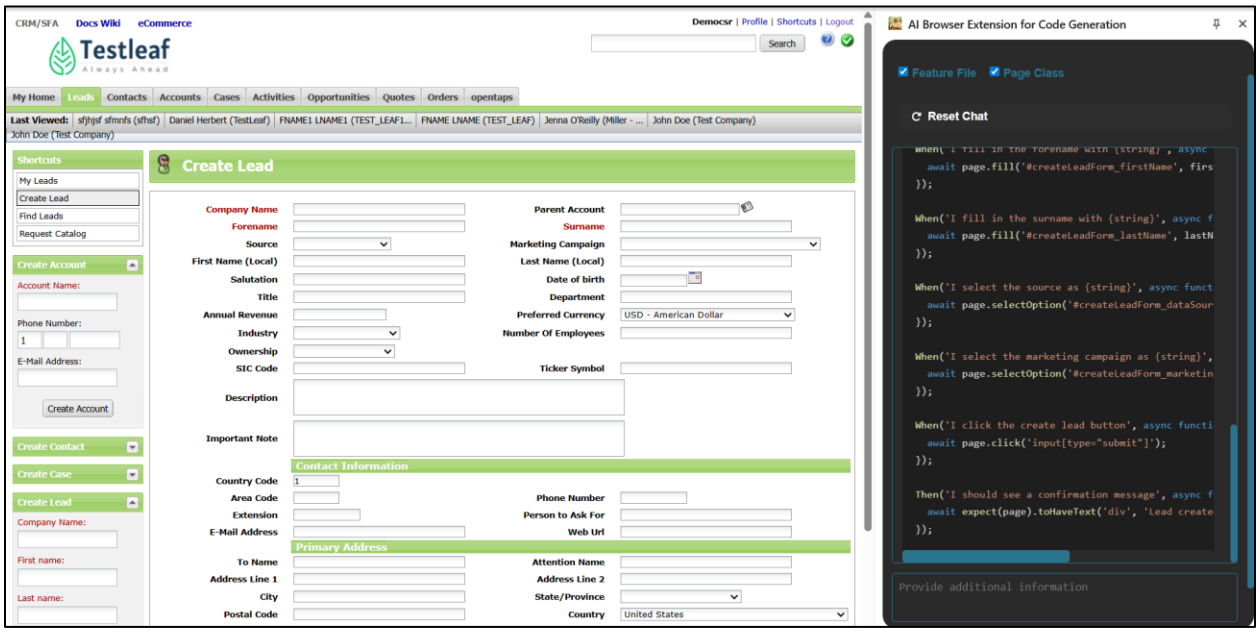
Step5: Selecting the fine tuned model - ft:gpt-4o-2024-08-06:testleaf-2::DW4E950A for for Playwright/Typescript code for leaftaps CreateLead functionality



Step6: Generating the Page class code in AI browser extension using the Fine tuned model for Leaftaps Create Lead functionality







<u>Step7</u>: Playwright/Typescript page class code with the fine tuned model - ft:gpt-4o-2024-08-06:testleaf-2::DW4E950A for leaftaps CreateLead functionality

Feature: Create Lead in CRM

Scenario Outline: Successfully create a lead with valid details

Given I navigate to the create lead page

```
  When I fill in the company name with "<companyName>"
  And I fill in the forename with "<firstName>"
  And I fill in the surname with "<lastName>"
  And I select the source as "<source>"
  And I select the marketing campaign as "<marketingCampaign>"
  And I click the create lead button
  Then I should see a confirmation message

 Examples:
  | companyName    | firstName | lastName | source     | marketingCampaign |
  | "TechCorp"     | "Arjun"   | "Reddy"  | "Cold Call" | "Affiliate Sites" |
  | "InnovaTech"   | "Priya"   | "Kumar"  | "Conference"| "Automobile"     |

import { Given, When, Then, Before, After } from '@cucumber/cucumber';
import { Page, expect } from '@playwright/test';
import { chromium, Browser } from '@playwright/test';

let browser: Browser;
let page: Page;

Before(async function() {
  browser = await chromium.launch();
  page = await browser.newPage();
});

After(async function() {
  if (page) await page.close();
  if (browser) await browser.close();
});

Given('I navigate to the create lead page', async function() {
```

```
  await page.goto('https://leaftaps.com/crmsfa/control/createLeadForm');
});

When('I fill in the company name with {string}', async function(companyName: string) {
  await page.fill('#createLeadForm_companyName', companyName);
});

When('I fill in the forename with {string}', async function(firstName: string) {
  await page.fill('#createLeadForm_firstName', firstName);
});

When('I fill in the surname with {string}', async function(lastName: string) {
  await page.fill('#createLeadForm_lastName', lastName);
});

When('I select the source as {string}', async function(source: string) {
  await page.selectOption('#createLeadForm_dataSourceId', { label: source });
});

When('I select the marketing campaign as {string}', async function(marketingCampaign:
string) {
  await page.selectOption('#createLeadForm_marketingCampaignId', { label:
marketingCampaign });
});

When('I click the create lead button', async function() {
  await page.click('input[type="submit"]');
});

Then('I should see a confirmation message', async function() {
  await expect(page).toHaveText('div', 'Lead created successfully');
});
```